



How to get predictions with predict_generator on streaming test data in Keras?

Asked 6 years, 6 months ago Modified 2 years, 7 months ago Viewed 77k times

In the [Keras blog on training convnets from scratch](#), the code shows only the network running on training and validation data. What about test data? Is the validation data the same as test data (I think not). If there was a separate test folder on similar lines as the train and validation folders, how do we get a confusion matrix for the test data. I know that we have to use scikit learn or some other package to do this, but how do I get something along the lines of class wise probabilities for test data? I am hoping to use this for the confusion matrix.

machine-learning python deep-learning keras confusion-matrix

Share Edit Follow Flag

edited Aug 24, 2017 at 22:03

Pieter

961

6

19

asked Sep 7, 2016 at 15:14

Raghuram

1,032

3

13

30

- 1

▲

▼

You can call the `model.predict_generator(...)` function with a generator that reads data from a directory containing the test set. It returns the predictions, which you can use to calculate a confusion matrix. Is that what you're looking for? See here for docs: [keras.io/models/sequential](#) – [stmax](#) Sep 7, 2016 at 15:30
- 1

▲

▼

Yes, I did see that, `predict_generator` returns a list of predictions which is a list of float values between 0 and 1. How do I interpret this? It cannot be directly used with the confusion matrix – [Raghuram](#) Sep 7, 2016 at 15:34 ✓
- 2

▲

▼

I haven't tried `predict_generator` yet (it's rather new), but it seems to return class probabilities. Try to convert values ≤ 0.5 to 0 and > 0.5 to 1. Once you have a list consisting of 0s and 1s you can feed it to the function for calculating the confusion matrix – [stmax](#) Sep 7, 2016 at 18:44
- 2

▲

▼

As an aside, this will work fine for two class problems, but what if there are more than two classes? – [Raghuram](#) Sep 8, 2016 at 0:32
- 1

▲

▼

If there are more than two classes, your network needs more than one output. For n classes you have n outputs and you predict the class that has the highest output. Have a look at the softmax function ([en.wikipedia.org/wiki/Softmax_function](#)) – [stmax](#) Sep 8, 2016 at 7:46

3 Answers

Sorted by: Highest score (default)

To get a confusion matrix from the test data you should go through two steps:

1. Make predictions for the test data

For example, use `model.predict_generator(...)` to predict the first 2000 probabilities from the test generator.

```
generator = datagen.flow_from_directory(
    'data/test',
    target_size=(150, 150),
    batch_size=16,
    class_mode=None, # only data, no labels
    shuffle=False) # keep data in same order as labels

probabilities = model.predict_generator(generator, 2000)
```

2. Compute the confusion matrix based on the label predictions

For example, compare the probabilities with the case that there are 1000 cats and 1000 dogs respectively.

```
from sklearn.metrics import confusion_matrix

y_true = np.array([0] * 1000 + [1] * 1000)
y_pred = probabilities > 0.5

confusion_matrix(y_true, y_pred)
```

Additional note on test and validation data

The Keras documentation uses three different sets of data: training data, validation data and test data. Training data is used to optimize the model parameters. The validation data is used to make choices about the meta-parameters, e.g. the number of epochs. After optimizing a model with optimal meta-parameters the test data is used to get a fair estimate of the model performance.

Share Edit Follow Flag

edited Mar 11, 2018 at 12:19

Pieter

961

6

19

- 3

▲

▼

Thanks for the code snippets. Could you link those two? In your example `y_true` seems to be populated with dummy data. Would you use `generator.classes` to populate the array? – [Gegenwind](#) Mar 10, 2018 at 14:10
- ▲

I'm not certain, but I think instead of `np.array([0] * 1000 + [1] * 1000)` you can get the same array by doing `generator.classes` – [Mehdi Nellen](#) Jan 2, 2019 at 13:28
- ▲

Does `model.predict_generator` automatically run test phase (ie, deactivate dropout)? – [Black Jack](#) 21 Apr 24, 2020 at 9:22

Here is some code I tried and worked for me:

```
pred= model.predict_generator(validation_generator, nb_validation_samples //
batch_size)
predicted_class_indices=np.argmax(pred,axis=1)
labels=validation_generator.class_indices
labels2=dict((v,k) for k,v in labels.items())
predictions=[labels2[k] for k in predicted_class_indices]
print(predicted_class_indices)
print(labels)
print(predictions)
```

You can then use:

```
print (confusion_matrix(predicted_class_indices,labels))
```

Make sure you use `shuffle=False` in your test generator (in my case it's validation generator) and reset it using `validation_generator.reset()` before you make your predictions.

Share Edit Follow Flag

edited Aug 20, 2020 at 23:10

Light X

3

2

answered Jul 1, 2019 at 11:02

rsalhd

31

1

For confusion matrix you have to use sklearn package. I don't think Keras can provide a confusion matrix. For predicting values on the test set, simply call the `model.predict()` method to generate predictions for the test set. The type of output values depends on your model type i.e. either discrete or probabilities.

Share Edit Follow Flag

answered Sep 7, 2016 at 16:25

enterML

3,001

9

26

38

- ▲

▼
- Thanks for the answer. I do know that Keras doesn't have its own confusion matrix package. My question is that `model.predict_generator` returns a list of float values which cannot be used to compute the confusion matrix. – [Raghuram](#) Sep 7, 2016 at 17:00
- ▲

▼
- What kind of data are you experimenting on? – [enterML](#) Sep 7, 2016 at 18:28
- ▲

▼
- I am working on images. – [Raghuram](#) Sep 8, 2016 at 0:31